

RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer

Mehrad Haghshenas[✉]

University of Waterloo

David R. Cheriton School of Computer Science

Waterloo, Canada

m3haghsh@uwaterloo.ca

Meng Xu

University of Waterloo

David R. Cheriton School of Computer Science

Waterloo, Canada

meng.xu.cs@uwaterloo.ca

Abstract

Conformance testing asks whether an implementation agrees with its specification. When the specification is prose, one established response is to mechanize it: an executable specification serves as the oracle, and an input on which an implementation disagrees with it is a candidate bug — in the implementation, or in the mechanization itself. How those inputs are obtained matters, because an oracle only adjudicates what it is given. They can be derived from the specification itself, and coverage-guided generators drive aggregate coverage high. But coverage is an aggregate over the specification, not a way to name one condition and obtain an input that reaches it. That matters for error conditions in particular, where implementations are known to diverge and where the code is empirically under-tested.

This paper presents RuSMT, a framework built around a domain-specific language whose types denote Z3 sorts and whose primitives denote Z3's operations. A program in the DSL is an executable specification. The author marks a branch with a named marker; in this paper, this always indicates an error condition. The single source is read two ways: compiled by `rustc` it executes as the conformance oracle; lowered to SMT-LIB it yields one reachability query per marker. A satisfying model yields the input, which a printer renders as source in the language being specified. That source is the test. An `unsat` says that no input reaches the marker. When the solver returns unknown or times out, RuSMT turns to a language model: it reads the emitted SMT-LIB and Z3's earlier verdicts and returns one concrete candidate, pinned into the unmodified query as a single equality. The proposer shapes the search and Z3 decides. A rejected candidate is fed back to the model, and the loop repeats up to a fixed number of rounds.

We write two executable specifications in the DSL: IMP, Winskel's canonical small imperative language, as a demonstration, and a TOML 1.1.0 parser, written from the published specification. We mark 2 branches in the first and 183 in the second. Z3 alone answers both IMP queries, in under a quarter of a second. It answers none of the 183 TOML queries at any budget we could afford: lowering a recursive parser whole-program is bounded model checking, whose formula-size blowup is well known. With the language model in the loop, RuSMT reaches 146 of the 183; the 37 misses are named.

We ran the TOML generated test suite against four independent TOML parsers; on 12 of the inputs the parsers and our oracle did not all agree. 9 of these test behaviour the specification leaves open, such as integer width and float exponent range, so they are not conformance obligations at all. Three are implementation defects, and 1 of those was previously unreported: `smol-toml` accepts an array-of-tables header closed by a single bracket, which we have reported upstream.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; *Formal software verification*; • **Theory of computation** → *Automated reasoning*.

Keywords

executable semantics, conformance testing, differential testing, SMT, bounded model checking, large language models

ACM Reference Format:

Mehrad Haghshenas and Meng Xu. 2026. RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer. In *Proceedings of the 1st International Workshop on Specification-Driven Development Life Cycle (SpecOps '26)*, October 4–9, 2026, Oakland, CA, USA. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3842652.3843196>

1 Introduction

Conformance testing asks whether an implementation agrees with its specification. When the specification is prose, one established way to make the question mechanical is to write the specification as a program. K [25], PLT Redex [9], and Ott/Lem [21, 29] are built on that idea. The executable specification is then the oracle: run it and an implementation on the same input, and a disagreement is a candidate bug, in the implementation, or in the mechanization if it is not faithful to the prose.

An oracle only adjudicates the inputs it is given, so generating the inputs is the next step. They can be generated from the specification itself [22], and coverage-guided generation drives aggregate coverage high [15]. But coverage is an aggregate over the specification: it does not let an author point at one condition the specification states and obtain an input that reaches it. That is particularly relevant for error conditions, where the code is empirically under-tested: in a study of production failures in distributed data-intensive systems, 92% of catastrophic failures resulted from incorrect handling of non-fatal errors, and 58% could have been caught by simple tests of the error-handling code [40]; error paths are routinely dropped in Linux file systems [10]. It is also where implementations diverge.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SpecOps '26*, Oakland, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2970-6/2026/10

<https://doi.org/10.1145/3842652.3843196>

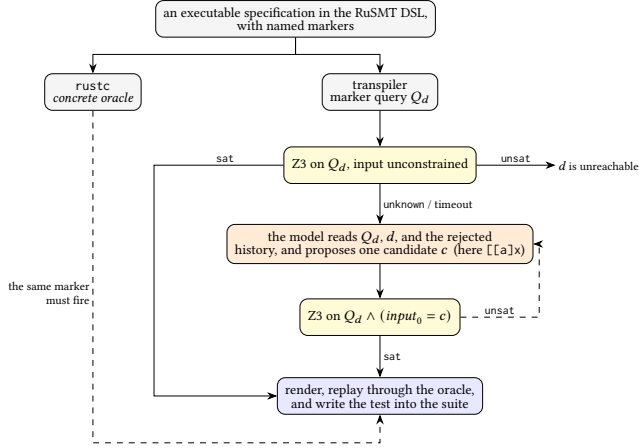


Figure 1: One executable specification, two readings, and the loop that turns one marked branch into one test, shown with the running example of §3.

No two JSON parsers agree across malformed documents [28], discrepancies between parsers on ill-formed input are a documented source of security vulnerabilities [4, 27], and a conformance suite asserts error behaviour normatively [8].

RuSMT is built for the case where the author wants to name the condition and obtain an input that reaches it. The specification is written in a subset of Rust whose types denote Z3 sorts and whose primitives denote Z3’s operations [7]; compiled, it is the oracle, and lowered by the transpiler, it is what the solver reads. The author marks each condition to be covered (in this paper, always an error condition) with a named marker, and the transpiler emits one reachability query per marker, so the suite has one input per condition and a condition that is not reached is reported by name. Z3 is asked each query. Where it returns a model, that model is the input; where it returns no verdict, a language model proposes a concrete input, Z3 checks the proposal against the unmodified query, and a rejected proposal is fed back to the model, for a fixed number of rounds. No input becomes a test except by a solver verdict. Figure 1 shows the flow; §2–§4 give the details.

Contributions. This is a tool-and-artifact paper. We propose no new solver, synthesis algorithm, or completeness result; the ideas it composes — conformance testing from a formal specification [12], solver-aided synthesis [30], and counterexample-guided inductive synthesis with a language model as the candidate generator [13] — precede it.

- (1) A framework in which one recursive Rust interpreter is both the conformance oracle, by compilation, and the source of one reachability query per marked branch, by lowering to SMT-LIB, and in which every generated test is admitted by Z3’s verdict on the query of the marker it is filed under (§2–§4).
- (2) Two executable specifications written in it: IMP, with 2 markers, as a demonstration, and a TOML 1.1.0 parser with 183, written from the published specification. Z3 alone answers both IMP queries and none of the TOML ones; with the language model in

the loop the TOML suite covers 146 markers, and the 37 misses are named (§5, §6).

- (3) A run of the TOML suite against four independent parsers. On 12 inputs the parsers and our oracle did not all agree: three are implementation defects, 1 of them previously unreported and now filed, and 9 test behaviour the standard leaves open, so they are not conformance obligations (§6).
- (4) A negative result on solver-alone synthesis. Lowering a recursive parser whole-program is bounded model checking [3, 6]; we report how sharply its known blowup arrives on a real parser, and what the working generator turned out to be (§3, §6).

Research questions. RQ1: How many of the marked branches does the pipeline produce a test for, and how many of those does the solver produce alone? RQ2: How often does a proposed input fail to reach the marker it was proposed for — that is, how many tests would have been mislabelled without the solver’s verdict? RQ3: Run against independent implementations, on how many inputs do the implementations and the oracle disagree, and what does each disagreement turn out to be?

2 The RuSMT DSL

A specification in the RuSMT DSL is a Rust program that is read in two ways. When it is compiled by rustc, it runs as an ordinary interpreter; when it is lowered by the transpiler, it denotes an SMT formula. For these two readings to agree, every construct that the DSL admits must have a meaning that the solver can express. That requirement bounds the subset from above; what bounds it from below is that the subset must still be enough to write an interpreter or a parser in, i.e., recursive functions over algebraic data types, sequences, and numbers. The DSL sits between the two bounds: it admits the constructs of Rust for which the transpiler has an SMT meaning and rejects the rest, and those constructs sufficed for both case studies of §5. We write parsers and interpreters in it, although nothing in the framework is specific to either. In this section we describe the fragment, the standard library that gives it its types, and the marker type on which the synthesis pipeline is built.

The fragment. The parser of the transpiler accepts a fixed set of expression forms and rejects every other Rust expression with an *invalid expression* error. The accepted forms are literals (integer, floating-point, boolean, and string), variables, the construction of a struct, of an enum variant, or of a tuple, field access, function calls and method calls, if with a mandatory else, match, blocks and parenthesized expressions, the dereference * of a Boolean (which is how a branch condition is read), and three macros, namely the bounded quantifiers forall!, exists!, and choose! over a collection. A function body consists of let bindings followed by a single expression. Consequently, there are no loops, no assignments, no closures, no references, and no indexing; recursion, including mutual recursion, is the only form of iteration. Each of these exclusions preserves a property that the lowering depends on. Without mutation and references, a value never aliases and never changes under another name, so a let is a substitution and the translation is referentially transparent; this is the same discipline that lets a big-step operational semantics be read as a system of equations.

Likewise, without loops, repetition is expressed as a recursive definition, which is also how a solver expresses it. The interpreters we wrote read much like the rules they formalize; Winskel’s big-step rules for IMP [36] transcribe almost line for line.

At the type level, `#[smt_type]` accepts a struct with named or tuple fields and an enum whose variants may be unit, tuple, or struct variants; it rejects unit structs, default values, and any other attribute, and it derives `Copy` (among other traits) for the type, so that values cannot alias. Type parameters are permitted provided that they are bounded only by the library’s SMT trait, and the transpiler monomorphizes generic functions on demand. At the function level, `#[smt_fn]` rejects `const`, `async`, `unsafe`, an explicit ABI, and variadic parameters. Accordingly, a specification outside the fragment fails to compile rather than being mistranslated.

Errors as values. The fragment has no `panic!`, `?`, or `.unwrap()`, so every departure from the expected path that a specification expresses is a returned value (for instance, a `ParseResult::Err` carrying a `Path`) rather than an exception, and each such branch is where a marker is placed. A DSL function can nevertheless panic at run time, through a library primitive that is called outside its documented precondition; the rule for the standard library given below governs that case, and the interpreter is expected to guard such calls.

The standard library. The types of the DSL denote Z3 sorts, and their methods denote Z3 operations; Table 1 lists them. The scalar types map directly onto Z3 theories, namely `Boolean`, the unbounded `Integer` and `Real`, the bit-vectors `I32`, `I64`, `U32`, and `U64` (where signed and unsigned are two readings of one Z3 sort, distinguished by the operations applied to it), and the IEEE floating-point types `F32` and `F64`. `String` and `Seq<T>` map onto the sequence theory. `Set<T>` and `Array<K, V>`, however, are not the SMT array theory directly, since that theory has no notion of size; each is lowered to an algebraic datatype that pairs an SMT array with an explicit cardinality (and, for `Array`, with a second array recording which keys are present), so that `length` and `is_empty` become definable. `Path` is the marker type and is described below. `Cloak<T>` is a Box-like indirection that exists only so that an author can write a self-referential type, e.g., `Add(Cloak<Aexp>, Cloak<Aexp>)`, under Rust’s sized-type rule; the IR erases it, lowering every `Cloak<T>` to `T`, because Z3 supports recursive datatypes natively.¹ Every method in the table is registered in the transpiler as an *intrinsic*, i.e., a fixed mapping from a method call to the SMT term that the backend emits for it, and this registry is the only place where a Rust method acquires an SMT meaning.

Two layers are thus kept apart. The standard library denotes Z3’s theories and nothing else, whereas the interpreter encodes the semantics of the target language on top of it. The particularities of a target (two’s-complement wraparound, a parser’s error conditions, and so on) live in the interpreter as explicit guards, and never in the library. Accordingly, the correspondence obligation of §4 is argued once, over a fixed vocabulary, rather than once per case study.

The rule that governs this vocabulary is where a framework of this kind is most tempted to cheat; therefore, we state it exactly. A

Table 1: The standard library: DSL types, the Z3 sorts they denote, and their operations.

Type	Z3 sort	Operations
Boolean	Bool	and, or, not, xor, implies, iff, ite
Integer	Int	arithmetic, comparisons, radix decoding (from_hex_str, ...), range tests (is_gt_i64_max, ...), conversions
Real	Real	arithmetic, rounding, comparisons, conversions
I32, I64, U32, U64	(_ BitVec 32), (_ BitVec 64)	bv_add, ..., bv_rem, comparisons, bitwise operations, shifts, rotations
F32, F64	(_ FloatingPoint 8 24), (_ FloatingPoint 11 53)	IEEE arithmetic, comparisons, predicates, rounding, conversions
String	String	at, concat, contains, starts_with, index_of, substr, replace, to_int, from_int, to_code, from_code, ...
Seq<T>	(Seq T)	at, extract, concat, append, unit, contains, prefix_of, suffix_of, index_of, replace, length
Set<T>	datatype over (Array T Bool) with cardinality	insert, remove, contains, is_subset, is_disjoint, length
Array<K, V>	datatype over (Array K V) with presence and cardinality	store, select, del, contains_key, length
Path	(_ BitVec N), one bit per marker	named, merge
Cloak<T>	erased to T	shield, reveal

primitive denotes Z3’s operation, not Rust’s and not the target’s, and where the two readings differ, the library documents the difference instead of hiding it. For a primitive f , exactly one of the following holds.

- (1) **Total agreement.** Rust and Z3 agree on every value of the sort, and f is used directly.
- (2) **Agreement under a precondition.** They agree on a documented subset of the values and differ outside it, and the difference takes one of two forms. In the first, Z3 defines a value where Rust panics: division by zero is defined in SMT-LIB (bvdiv returns all ones, and bvdiv returns all ones for a non-negative dividend and 1 for a negative one), whereas the corresponding Rust division panics. In the second, the SMT term is well-formed but the theory does not fix its value: $(\text{div } x \ 0)$ over the integers is a legal term, and with $x = 7$ Z3 satisfies $(= (\text{div } x \ 0) \ 100)$ and $(= (\text{div } x \ 0) \ 999)$ alike.² In either form, the *interpreter* guards the differing inputs and calls f only on the rest.
- (3) **No total Rust reading.** Z3 admits values that Rust cannot represent, and the primitive is not offered at all.

Listing 1 is the division guard of the IMP arithmetic evaluator. The interpreter branches on a zero divisor because that is the condition the specification names and the input we want to synthesize, so the marker sits on that branch; the other branch calls `bv_div`, which agrees with Z3 on every nonzero divisor. It is ordinary recursive Rust in which the only token specific to synthesis is `Path::named`. From this source, RuSMT synthesizes $A := (0 / 0)$, and replaying the program through the same `eval_aexp` reaches the marker.

```

if *new_r.eq(I64::from(0)) {
  ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
  ArithmeticResult::Ok(new_l.bv_div(new_r))
}

```

¹Provided that the type also has a non-recursive variant; otherwise the lowered datatype is uninhabited.

²Both checked on Z3 4.15.4. The second form is the more dangerous one: if such a term is left unguarded, the solver is free to choose its value so as to satisfy the marker assertion, and the resulting model corresponds to no real execution.

Listing 1: The division guard of IMP’s evaluator (operands already evaluated to `new_l` and `new_r`; Err propagation elided).

Markers. A *marker* names a program point that the author wants covered. Concretely, a *Path* is a set of marker ids; an execution that trips several markers accumulates all of them, and *Path* : *merge* is set union. Two properties of this type carry the method, and we discuss them in turn.

The first property is identity. *Path* : *named*("d") derives its id as a fixed hash of the name *d*, and the same function computes this hash in the transpiler and in the compiled evaluator. Hence, the id that the query asserts and the id that a replayed witness carries are identical by construction, which is what allows replay to check that the *named* marker fired, rather than merely some marker.

The second property is representation. In SMT, a *Path* is a bit-vector with one bit per named marker in the program, and the set operations of the type become bit-vector operations:

$$\begin{aligned} \emptyset &\mapsto (_ \text{ bv } 0 \text{ N}), & \{m\} &\mapsto \text{one-hot}(m), \\ l \cup r &\mapsto (\text{bvor } l \text{ } r), & T \subseteq s &\mapsto (= (\text{bvand } s \text{ } M_T) \text{ } M_T), \end{aligned}$$

That is, the empty path is the zero vector, a named marker sets its own bit, merge is bitwise or, and “the path *s* contains every marker of the target *T*” is the masked equality on the right, where *M_T* is the vector with exactly the bits of *T* set. The encoding is exact, in the sense that union and membership are represented as themselves, so no query is answered from information that the encoding has lost. Bit indices are derived from the ids and never leave the query they were emitted into; the two readings exchange the name, and never the number.

3 From a Marker to a Conformance Test

The annotated source is compiled in two modes. In oracle mode, `rustc` compiles the interpreter, which then executes on concrete inputs. In solver mode, the transpiler parses the fragment of §2, lowers it to a sort-checked intermediate representation by Hindley–Milner unification over SMT sorts, monomorphizes generic functions on demand, records every *Path* : *named* site as a target, and emits one query per target. Each query is a self-contained SMT-LIB2 file, which is what makes every result in this paper re-runnable with a bare `z3 -smt2` on a committed file. The driver then runs the loop of Figure 1 over these files and writes the accepted witnesses out as the suite. Adding a language means writing an interpreter and registering one oracle entry for replay; it touches neither the front end nor the backend, which is what IMP and TOML exercise.

3.1 Markers to Queries

RuSMT uses the solver to ask one question: does there exist an input such that the executable specification reaches marker *d*? For a target *T*, which is a set of marker ids, the query declares one constant per parameter of the entry function (`input_0`, ...), bounds any code-point parameter to the alphabet of the solver (see §8), applies the entry function, and asserts that the *Path* carried by the returned error value *contains* every id in *T*, guarded by the constructor test that selects the error variant; it ends with

(`check-sat`) and (`get-model`). In other words, the query is

$$\exists x. \text{entry}(x) = \text{Err}(p) \wedge T \subseteq p,$$

where *x* is the object-language program or input text and the containment is the masked equality of §2. One formula thus covers both shapes that a target can take, namely the singleton *T* = {*i*} of an ordinary *Path* : *named* and the multi-marker *T* of a *Path* : *merge* site, which asks for one run that reaches all of them. Containment rather than equality is what keeps the assertion robust to a run that fires *additional* markers, which accumulating error handling makes routine.

A recursive interpreter is not a finite formula, and the *k=N* flag selects between two routes. With *N*=0, which is the default and what both case studies use, the recursive strongly connected components of the call graph are emitted as `define-funs-rec`, and the solver handles the recursion natively. With *N* ≥ 1, every recursive component is unrolled into *N*+1 depth-indexed copies *f*₀, ..., *f_N*, where *f_d* calls *f_{d-1}* for callees in the same component, *f*₀ is a terminator, and external callers enter through *f_N*. Under depth-*N* unrolling, a sat model is a witness that is reachable in at most *N* recursive steps, and this is the sense in which the completeness is *bounded*, as in bounded model checking [3].

Unrolling cuts both ways, and IMP shows both directions. At *k*=1, the division-by-zero target returns `unsat`, whereas the same marker is `sat` with a replay-accepted witness at *k*=0. A bounded `unsat` therefore means only that there is no witness within depth *k*, and never that the marker is unreachable; for this reason, the pipeline treats `unsat` as a genuine verdict only at *k*=0. In the other direction, the terminator *f*₀ has to return something, and what it returns is a nullary *bottom* variant that the concrete semantics never produces, so that the cutoff cannot satisfy an *Err*-marker assertion by itself; replay would reject any such model in any case, since the bottom value never arises in a concrete run.

3.2 Solving, and Where the Solver Stops

The query is first handed to Z3 exactly as emitted, with the input unconstrained. For IMP, this is the whole story; both markers are discharged unaided, in 220 ms and 109 ms respectively. We treat Z3 as sound for the verdicts it returns and as an incomplete search procedure otherwise, so an unknown or a timeout costs coverage and nothing else.

For TOML, the solver never once succeeded, and it is worth being precise about the reason, since it is not specific to us. Emitting the entire semantics as one formula and asking whether some input satisfies it is bounded model checking [6]. BMC unrolls loops and recursion a bounded number of times and encodes every path within that bound into a single query, and the resulting growth of the formula is a documented consequence of the approach [3, 6]. Our encoding takes the sharpest form of it. The transpiled parser functions are non-recursive `define-fun` definitions, which by construction are eliminated by macro expansion [2], and this expansion happens during preprocessing, i.e., before the solver can know which branch is taken. A concrete document lets the simplifier fold each conditional as it expands; a symbolic document leaves both arms of every conditional alive, and the expansion multiplies through the nesting.

What we can add is the shape of this effect on a real artifact, and it is sharper than the general statement that symbolic inputs are expensive would suggest. Taking the query of one marker and varying only how much of a 16-character document is left free, the fully pinned query decides in 798 ms, whereas a single free character is already enough to return unknown at 30 s; two, three, four, six, and eight free characters behave identically. A single character ranges over roughly 55k values, so this is not an effect of the size of the search space. It is the expansion: any free variable at all defeats the folding, and the whole parser stays live regardless of how tightly that variable is constrained. Consequently, there is no partially symbolic middle ground to exploit here; the concretization step that makes concolic testing work [5, 23] has no foothold, because the formula never becomes small. We also tested emitting everything as `define-fun-rec`, so that unfolding becomes lazy; this made matters worse, since the character-class predicates could then no longer fold either.

3.3 The Language Model in the Loop

The design of the loop follows from this diagnosis. If the solver is cheap on a determined input and hopeless on a symbolic one, then the productive move is to determine the input by other means and let Z3 do what it does well. Accordingly, when Z3 returns unknown or times out, the driver invokes a language model, which receives the emitted SMT-LIB text, the name of the marker, and the candidates already rejected together with Z3's verdict on each, and which returns one candidate. The pipeline appends a single equality, $input_0 = c$, to the *unmodified* query. A sat answer is then a model of the original marker query, whereas an unsat answer says only that this particular candidate does not reach this particular marker, and it is fed back as the context of the next round. Each marker is given the same number of rounds, and running out of rounds is a spending limit and nothing more.

Rather than returning a bare unsat to the model, the pipeline puts a second query to Z3, with the same input pinned, the marker assertion dropped, and the returned Path bound to a constant that the query reports; the answer names the marker that the candidate actually reached. The running example of Figure 1 shows why this matters. The TOML reference raises `array_table_missing_close_char` when an array-of-tables header is closed by something other than `]]`. Z3 returned no verdict on the unconstrained query for this marker, as for every other TOML marker, so the model was shown the SMT-LIB, the name, and an empty history, and it proposed `[[a`. Pinned into the query, that document was unsat; the second query reported that it reaches `array_table_missing_close_eof` instead, since the header runs into the end of the input before any closing bracket. That answer was fed back, and in the next round the model proposed `[[a]x`, which Z3 accepted in 1.4 s. The document was then re-executed through the compiled semantics, where it fired the same marker, and it entered the suite. In §6, one of the four TOML parsers we test accepts this document as valid.

Two aspects of this arrangement need to be stated explicitly. The first is what the model is not allowed to see. It runs in an empty working directory under an operating-system sandbox that denies reads of the repository, so it cannot read `lang/src/toml/`

and recover an answer from the Rust source. The denial is at the operating-system level on purpose, because the read-only mode of the model runner forbids writes but permits reading any file on disk, and it would therefore not have prevented this. We verified the result rather than assuming it; asked to quote a named function from the reference parser, the model replies that it has no file access. The claim that the model reasons from the emitted SMT-LIB depends on this, and hence we state it as a configuration requirement.

The second is that nothing in the design requires a language model. The proposer is an ordinary shell command that reads a prompt on standard input and writes a candidate on standard output, so an enumerator or a fuzzer is a drop-in replacement, and the acceptance step, and with it the guarantee of §4, is unchanged by the substitution. We did not try an undirected generator; the numbers of §6 show that Z3 rejected about three in four proposals even from a marker-targeted one, mostly because an earlier error masked the intended one, and a generator with no notion of the marker faces a strictly harder version of that problem.

3.4 Rendering a Model

A sat model is rendered back to object-language source by a printer, which extracts `input_0`, inlines `let`-shared subterms, maps managed constructors back to AST nodes, canonicalizes solver-chosen identifiers, and decodes bit-vector literals as two's-complement signed integers. Canonicalizing identifiers is sound here because which marker fires does not depend on their spelling. This rendering step is visible in IMP, where Z3 returns a Com AST that has to be printed as an IMP program; in TOML, the top-level input is already a sequence of Unicode code points, so the decoded model is the document itself.

4 What RuSMT Asks the Reader to Trust

RuSMT is a trusted computing base in the same sense that the kernel of a proof assistant is one, at a much smaller scale. If the framework is accepted, then the conformance tests it produces are tests whose acceptance was decided by a solver rather than by a person reading them, and we state here exactly what that acceptance rests on.

The trusted base. Four things are trusted. First, the RuSMT specification itself; the transcription of a prose standard into the DSL is done by a human and is not verified against the standard. Second, the correspondence between the standard library and Z3, which we discuss next. Third, the transpiler that lowers the DSL to SMT-LIB. Fourth, Z3. The concrete parser and pretty-printer used for replay are trusted as well. A bug in any of these is a bug in our results.

The per-construct correspondence. Everything else rests on one obligation, stated for every standard-library primitive.

Definition 4.1 (Per-construct correspondence). For every stdlib function f and every concrete input x satisfying the documented preconditions of f ,

$$rust_impl(f, x) = z3_eval(z3_formula(f), x).$$

We *assume* this rather than prove it; what we do instead is make it auditable. The documentation of each stdlib method states the exact Z3 formula that the backend emits for it, together with its preconditions and, where the two readings genuinely

differ, the inputs on which they differ. Such divergences exist and are documented, e.g., `String::to_int` on a non-numeric string, `String::from_int` on a negative integer, `replace_all` with an empty pattern, and `Array::select` on an absent key. In every such case, the obligation is pushed onto the author of the interpreter, who must guard the input before the call. The TOML parser, for instance, checks each character with `is_hex_digit` and strips prefixes and underscores before anything reaches `Integer::from_hex_str`, so that an invalid input becomes a named marker and never reaches the library at all. This is the division of labour of §2: a primitive that matches the target exactly is used directly, and a primitive that diverges on specific inputs is guarded on those inputs. Because concrete Rust and Z3 take the *same* branch on the same input, the equality of Definition 4.1 survives the conditional.

Which inputs are free. One point is easily overlooked, and it is load-bearing. The set of inputs that the SMT query treats as free must coincide with the set of inputs that the concrete oracle leaves free, and everything else must be fixed to the same value on both sides. The IMP entry point `eval_com(c: Com)` fixes the empty store internally, so the program is the single free input; had the store been left free, Z3 could have chosen a starting state that the concrete run never uses, producing a “witness” that does not replay. The TOML entry point `parse_toml` needs no such wrapper.

Why a sat answer is enough. This is the one place where something is proved, and it is elementary. A candidate enters the query as an added conjunct, so the strengthened query is $Q_d \wedge (\text{input}_0 = c)$. Adding a conjunct cannot create a model that Q_d lacked; hence, every model of the strengthened query is a model of Q_d . A sat verdict is therefore a statement about the original query, i.e., the one encoding the marker that the author named, and it remains so however the candidate was produced. Nothing about the model needs to be trusted for this reason, and it is also why we do not describe the model as “untrusted but checked”: there is no checking step in which it could be trusted or not, only a fact about conjunction.

Replay. A candidate is accepted for target d only if Z3 decides that the lowered semantics reach the named marker d , and the same candidate, re-executed through the concrete interpreter under a wall-clock bound, fires the same d rather than merely some marker. Since acceptance is already a logical consequence of sat, the only way an accepted candidate can be wrong is if Definition 4.1 fails on some construct that the candidate exercises. Replay is a check on the lowering, not on the proposal, and it catches precisely that disagreement. It also fails safe, in that a defect in the correspondence shows up as a spurious rejection and never as a false acceptance. Likewise, replay would reject a model that satisfied the assertion only at the cutoff of a depth-bounded unrolling (§3).

Completeness. We claim no completeness in any general sense. Even when an input reaching a marker exists, Z3 may return unknown or time out, since the encoded formula can fall outside what the solver decides within its resource limits. At best, the pipeline has bounded completeness relative to a depth bound, and even that is contingent on the solver.

5 Case Studies

RuSMT is meant for the authors of new executable specifications, and the two case studies were chosen to stress two different regimes rather than to maximize a benchmark count.

5.1 IMP

IMP is the canonical small imperative “while” language of Winskel [36]. Its big-step semantics is familiar and recursive, yet small enough that native Z3 recursion ought to succeed if the pipeline is sound at all. We added two dynamic error branches, namely `division_by_zero`, and `undefined_variable` for reading a variable from the initially empty store, so that the study answers whether a direct-style interpreter can be lowered, solved, rendered, and replayed without bounded unrolling and without a proposer. Note that Z3 here synthesizes an AST rather than text, so the printer of §3 is exercised as well. The specification is 1,084 non-blank lines of Rust.

5.2 TOML

TOML 1.1.0 [24] was chosen because it is large enough to defeat the solver while still having a precise public specification and several independent implementations to test against. The executable parser, written from the published specification, has a single free input, i.e., a sequence of Unicode code points, and 158 functions covering documents, key-value pairs, tables, arrays, inline tables, booleans, integers, floats, strings, and datetimes, in 5,878 non-blank lines. Its 183 named markers cover syntactic errors (malformed keys, unterminated strings, invalid escapes, and missing array or table delimiters) as well as semantic violations (integer and float overflow, out-of-range date and time fields, duplicate keys, and redefined tables). This is the case study that motivated the method, for the reasons given in §3. The witnesses that come out of the loop are small and readable, which is a property that we did not design for but which matters downstream, since a minimal document isolates the rule it violates; besides the running example, the run produced $d = 1979-13-01$ for the invalid-month marker and $a = [$ for an unterminated array. We do not claim a verified or complete TOML formalisation. We claim a formalisation with 183 named obligations, and a suite generated from it.

6 Evaluation

Setup. All measurements were taken on an Apple M1 running macOS 26 with Z3 4.15.4 and Rust 1.88.0, with Z3 driven as a subprocess over SMT-LIB2. Both suites were generated by the driver of §3, the TOML one through a resumable per-marker sweep with six workers, so that a stopped run does not repeat the model calls it has already paid for. The proposer was OpenAI’s gpt-5.4-mini, invoked through the codex command-line runner as an ordinary shell command, with one witness requested per marker, nine rounds per marker, and a 20 s Z3 budget for each pinned candidate. The unconstrained solve was given a budget of 2 s per TOML marker, for the reason given in §3.2: separate probing had established that a symbolic TOML input does not decide at any budget we could afford, and that Z3 charges its `-t`: limit against CPU time rather than wall-clock time, so that a 60 s budget on one of these queries returned unknown only after 347.6 s of wall-clock time. A larger

Table 2: New witnesses per round over the 183 TOML markers. Since a witness accepted in round k would have been accepted at any round limit $\geq k$, the cumulative column reads directly as coverage at each limit.

Round	New witnesses	Cumulative
1	57	57
2	56	113
3	19	132
4	6	138
5	3	141
6	2	143
7	0	143
8	2	145
9	1	146

budget therefore buys nothing and is subtracted from the rounds that do finish.

The round limit and the per-candidate budget are constants that we did not tune. A budget can only turn a marker that would have been covered into a miss, never manufacture a witness (§8), and rather than asserting that the limits were generous we report how binding they were: the cumulative column of Table 2 is a sensitivity curve over the round limit, recovered from the same run at no extra cost. Every model exchange is recorded and keyed by a content hash of the prompt, so the reported run replays without model access. Because the model is nondeterministic, re-running it from scratch is not guaranteed to regenerate an identical suite; the scientific object here is the recorded run together with the suite it produced, and not a claim of determinism.

RQ1: coverage, and the share of it that the solver produced alone. For IMP, Z3 returned a model for both markers under native recursion, in 220 ms and 109 ms, with no model involved; the printer rendered them as $A := (\emptyset / \emptyset)$ for `division_by_zero` and $v\emptyset := v\emptyset$ for `undefined_variable`, and both replayed. We do not run IMP through the differential harness, since it is a small end-to-end demonstration and not a language with competing production parsers.

For TOML, the lowering produces SMT-LIB that Z3 accepts without a front-end error, but the unconstrained solve produced no witness for any of the 183 markers, in the manner described in §3.2; no query returned `unsat` either, so no marker was shown unreachable. With the language model in the loop, coverage went from 0/183 to 146/183. Table 2 shows how the witnesses arrived. The first two rounds account for the majority of them, the later rounds add progressively fewer, and witnesses were still arriving in rounds eight and nine; the curve had not flattened at the limit. Consequently, every coverage figure we report is a lower bound at that budget, and the 37 markers without a witness are markers left open, not markers ruled out.

The misses are not spread uniformly, and their distribution is the most diagnostic part of this result. 20 of them are date, time, and numeric-offset markers; 10 are numeric-literal markers, mostly float overflow and malformed radix forms; 6 concern table or key redefinition and unterminated composite constructs; and 1 is an array case. The pattern is consistent. The loop handles a local syntax error well, and it struggles where the intended marker is easily masked by an earlier parse error, or where the input must first pass

through a long chain of numeric or temporal sub-parsers before it can reach the labelled branch. We did not fill the misses with hand-written tests; the accepted witnesses *are* the suite, and the artifact records the marker-level ledger, including every miss by name.

RQ2: how many tests would have been mislabelled. The per-round history was recorded for every marker. Over the run, the model made 588 proposals that reached Z3 and got a verdict: 146 were accepted and 442 were rejected as `unsat`. A further 28 proposals repeated an already rejected candidate and were dropped before Z3, and on 19 occasions the model returned no parsable candidate at all. No acceptance check timed out. In other words, about three of every four proposals did not reach the marker they were labelled with, and without the solver’s verdict the suite would have carried several hundred tests filed under the wrong obligation. This is also the answer to the objection that a language model could produce such a suite on its own: it could produce the documents, but not the labelling, since without the query there is nothing against which to check the claim that a given document exercises a given rule.

The second query of §3 tells us why the proposals failed. Over the run, 88.2% of the rejections came back naming a different marker that the document actually reached, 5.9% reported that the document was simply valid, and on 5.9% the observation itself returned no verdict, so the proposer received the bare `unsat` it would have had anyway. The failures are systematic rather than scattered. The usual cause is that the proposed document contains an earlier violation, so that the parser stops before the intended branch; one marker alone accounted for the largest share, the proposer having repeatedly written a bare value with no key = in front of it, which is exactly the kind of error that a counterexample can correct and that a rejection cannot.

The converse also needs to be said. On a fully pinned input, Z3 is deciding rather than searching, so on the TOML suite the model carries generation and the solver carries acceptance; what the solver supplies is the labelling, under the argument of §4 rather than under trust in the reference. Acceptance is itself substitutable: replaying the compiled reference agreed with Z3 on every candidate this run produced, so a replay-only pipeline would have yielded the same 146 inputs. We keep Z3 in the accepting position because it is what makes the argument a fact about conjunction rather than an appeal to the correctness of the reference, and not because it outperformed the alternative; the ablation that would settle the question (the same loop with the SMT-LIB withheld and the proposer given only the marker name and replay feedback) we did not run.

RQ3: what the suite found. Two findings live in this comparison, and they are not the same claim. An implementation that disagrees with the *oracle* is a conformance result: the executable specification says that the document violates a named rule and the implementation accepts it, so it is a candidate bug against the specification, which is only as good as our formalisation of it (hence the triage below). Implementations that disagree with *each other* are a differential-testing result [20], and it is the weaker but more robust one, since it shows that the generated suite discriminates between production parsers whether or not our reference is right.

We ran the 146 generated documents against four implementations, each the latest release at the time of writing: the Rust `toml`

Table 3: Agreement of each implementation with the RuSMT oracle on the generated TOML suite. Every suite input is rejected by the oracle, so a divergence is an acceptance.

Implementation	Agreement	Divergences
Rust toml 1.1.4+spec-1.1.0	144/146	2
CPython tomlib (3.14.7)	137/146	9
BurntSushi toml v1.6.0 (Go)	142/146	4
Node smol-toml 1.8.0	142/146	4

Table 4: The 12 inputs on which the parsers and the oracle did not all agree, and the 19 parser-level divergences they produced. Group A is not an obligation of the standard and group B is already tracked upstream, so only group C was filed.

Group	Inp.	Div.	Filed
A Standard leaves it open	9	16	no
B Implementation defect, tracked	2	2	no
C Implementation defect, unreported	1	1	yes
Total	12	19	1

crate 1.1.4+spec-1.1.0, the tomlib module of CPython 3.14.7, BurntSushi’s toml v1.6.0 for Go, and smol-toml 1.8.0 for Node. Every document in the suite is one that the RuSMT oracle rejects, so a divergence is an implementation that accepts it. Table 3 gives the per-parser counts. All four parsers agreed with the oracle on 134 of the 146 documents; on the remaining 12 inputs the parsers and the oracle did not all agree, and on 10 of those the parsers also disagreed with each other. Table 4 accounts for all 12 inputs, which fall into three groups.

Group A: the standard leaves it open (9 inputs). This is the largest group, and it contains no defect on either side. Six inputs are the tomlib module accepting integers outside the 64-bit signed range. TOML states that “implementations are free to support any integer size” [24], recommends at least 64-bit signed, and requires an error only when a value “cannot be represented losslessly”. Our reference uses i64, so 18446744073709551616 genuinely cannot be represented and we *must* error; the arbitrary-precision integers of CPython represent it exactly, so it need not. Both follow the text. The other three inputs concern our i32 bound on the float exponent, and here the standard is not merely permissive but silent: it says only that implementations are “free to support any precision level” [24], and it states nothing about exponent range, overflow to infinity, or underflow to zero. Where the specification mandates nothing, no behaviour can violate it. The finding is therefore not about these parsers but about our own marker catalog. 9 of the markers in the suite (15 in the catalog as a whole, once misses are included) name branches that exist because of a representation we chose, and not because the standard distinguishes them. They are legitimate branches of our semantics and useless as conformance obligations, and a suite that ships them will fail implementations for making a different legal choice. For this reason we report them separately instead of folding them into a divergence count.

Group B: tracked defects (2 inputs). BurntSushi accepts `a = { b = 1 } \n a.c = 2`, which extends an inline table

that the standard calls self-contained, and `a.b = 1 \n [a]`, where a header redefines a table that a dotted key created implicitly. Unlike group A, the standard is explicit in both cases. Both are also already known to the maintainers: the project’s `toml_test.go` skips `invalid/table/redefine-02`, `redefine-03`, and `invalid/inline-table/overwrite-02` beneath the comment “TODO: fix this; we allow appending to tables, but shouldn’t”. Our suite reached them from the specification rather than from the test corpus, which is a check on the method; nevertheless, they are not new, and we do not file them.

Group C: the unreported defect (1 input). `smol-toml 1.8.0` accepts an array-of-tables header that is closed by a single bracket. The suite document is the running example `[[a]]x`, which minimizes to `[[a]]` and is parsed as `{ "a": [{}]`, while `[[a]]x`, `[a]x`, `[[a]]x`, and `[[a]] x` are all correctly rejected, as is `[[a]]` by the other three parsers. The case appears neither in `toml-test` nor in the project’s own skip list, and the reason is instructive: `smol-toml` passes all five existing `toml-test` cases for unclosed table headers, every one of which places a line after the malformed header, so that none of them reaches the end-of-input path. A corpus can be passed in full and still leave a branch untested. The grammar of TOML 1.1.0 settles the case, since `array-table-close = ws %x5D`. 5D requires two brackets, and this is the one divergence that we reported upstream.³

We report this distribution rather than a headline count. One property of the suite made the triage possible at all, namely that every divergence arrived as a minimal document already labelled with the rule at issue, so that deciding which group it belonged to was a matter of reading one line against one clause of the standard.

7 Related Work

Solver-aided executable semantics. Rosette [32, 33] is the closest solver-aided host: a symbolic virtual machine compiles a language subset to constraints, while concrete evaluation strips away ordinary host code. RuSMT is narrower and more conformance-oriented; the user writes a reference interpreter, marks specification obligations, and receives tests that were accepted by a solver. SyGuS [1] and SemGuS [14] synthesize against a separately authored grammar or a semantics encoded as constrained Horn clauses, whereas in RuSMT the unknown is simply an input to the interpreter that the user has already written. Syntactic [17] runs in the opposite direction, synthesizing a formal semantics from a black-box interpreter, which is complementary to our source-to-SMT route.

Specification-derived testing. Executable-semantics frameworks such as K [25], PLT Redex [9], and Ott/Lem [21, 29] derive a tool suite from rewrite or inference-rule specifications, and RuSMT derives a narrower tool from direct-style Rust, namely conformance tests aimed at named markers. JEST [22] is the closest in purpose, using a mechanized ECMAScript specification as oracle, synthesizer, and conformance tester; our distinction is the acceptance of each test by a solver verdict on an author-named marker, and the explicit sound-but-incomplete split that the TOML case study

³Reported as `squirrelchat/smol-toml` issue #65; the two missing cases were also contributed to `toml-test` as pull request #205.

exposes. Grammar-driven generators such as Csmith [39] remain the application domain rather than the technique.

Conformance testing from formal specifications. Deriving a conformance suite from a formal specification is a mature line of work, and we claim no novelty for the goal; TGV [12], for instance, synthesizes conformance test cases from specifications of non-deterministic reactive systems. What differs here is the packaging rather than the principle: the specification is an ordinary Rust program rather than a separate formalism, the same source compiles to the oracle so that no second artifact has to be kept in step with it, and the coverage criterion is carried by markers written inline in the semantics instead of by a coverage model maintained alongside it.

Whole-program versus per-path encodings. Our unconstrained query places us on the bounded-model-checking side [6] of a long-standing division: one query for the whole program, unrolled to a bound, versus one query per explored path. Symbolic and concolic executors [5, 23] take the other side, and the difference in constraint size is the reason they do not hit the wall reported in §3.2; their dominant cost is path explosion rather than the difficulty of any single path’s constraint [5]. Recent work pairs language models with either side. Cottontail [34] drives concolic execution on structured inputs, and AutoBug [16] decomposes the analysis path-wise and asks a model to do the reasoning that a solver would. We differ from both in what the model is shown, namely the emitted SMT-LIB and the verdicts of the solver rather than the program.

Language models behind a solver. We claim no novelty for the propose-then-check pattern. It is CEGIS [30] with a language model in the generator position, as in Jha et al. [13]. Lemur [38] uses LLM-proposed verification structure, and AquaForte [18] has a model propose SMT instantiations that the solver then rechecks. Test-generation portfolios such as CodaMOSA [15] and Cottontail [34] likewise call a model only once conventional generation stalls. Our modest difference is where the checked obligation lives: the same executable specification is both the oracle and the synthesis source, and the accepted obligation is a stable named marker rather than generic coverage.

8 Limitations and Threats to Validity

A marker covers what the author wrote. A marker covers only an error condition that the author actually wrote; a violation that the standard implies but our reference omits is simply not marked. Hence, the TOML suite is marker-complete relative to our formalisation and not to TOML 1.1.0. In the other direction, a marker may name a branch of our semantics that the standard does not require an implementation to have. The differential run identified 9 such markers in the suite (15 in the catalog, once misses are counted), namely those bounding a float exponent by i32 and an integer by i64, and it identified them only because the four parsers disagreed with the oracle on exactly those inputs. There may be others that no implementation happens to contradict, and our coverage figures count them as obligations like any other; disagreement between independent implementations and the reference is, at present, the only detector we have for this class.

A miss is not a proof of unreachability. A marker reported as uncovered is one for which no input was *found*, not one for which none exists. Only an *unsat* on the unmodified, unbounded query would establish the latter, and the unconstrained solve returned no verdict on the TOML queries. We wrote every marker to be reachable and expect none of the 37 misses to be a dead branch, but the pipeline cannot confirm this, and we do not claim it.

One witness per marker. The reported run generated one witness per marker, and this is by design: a marker names one condition, and one input that reaches it exercises that condition. A rule that can be violated in several distinct ways should be given several markers rather than several witnesses, which is how the TOML catalog is organized; integer overflow, for instance, is one marker per radix and sign. The driver can nevertheless request more witnesses per marker, at the cost of one blocking assertion and a re-solve each; we did not, in order to bound the number of model calls across 183 markers.

The authoring cost. The author writes one Rust program, of 1,084 non-blank lines for IMP and 5,878 for the TOML reference, and marks the error branches in it; from that single source, the framework derives the oracle, one query per marker, and the suite. The cost is therefore that of writing the executable specification once. It is real, and it is what the authoring mode below is aimed at.

Authoring the specification with a model. The framework also contains an authoring mode, in which a proposer drafts a specification in the DSL from a prose standard. Each draft passes through the same front end and backend as a hand-written one, through a reachability check (every declared marker must be reachable under the draft’s own semantics), and through a set of author-supplied examples, with every failure fed back to the proposer; the propose-and-check loop follows the pattern of AutoSpec [35] and SpecGen [19]. Its evaluation is future work, and the experiment is designed as follows. The proposer is given the integer section of the TOML standard and the DSL, and neither our reference nor our marker names, since all but one of the 57 integer marker names contain an explicit rule word and would give away what is being measured. The draft is then scored on two quantities that do not depend on names: accept/reject agreement with the reference over the invalid documents of this run together with a set of valid ones, and the number of distinct markers that the draft raises across the invalid documents, which asks whether a fine-grained error taxonomy is recoverable from prose. The harness is in the artifact.

The solver frontier. The dominant limitation is the whole-program encoding. Because the entire semantics is lowered into one query, every marker inherits the growth of BMC, and the measurement of §3.2 shows that the penalty arrives with the first free variable. The remedy used by concolic and dynamic-symbolic tools [5, 23] is to explore one path at a time, handing the solver only the branch conditions of that path. For RuSMT, this means a second backend: a symbolic walker over the syntax tree of the DSL, which remark already produces, that carries a path condition and forks at each conditional. The misses indicate where it would pay, since 10 of the 37 are numeric-literal markers whose documents are structurally trivial and whose difficulty lies in a numeric value

that the proposer has to guess and that a solver, given the path, could compute.

The synthesis alphabet stops at U+2FFFF. The Unicode Strings theory of Z3 is defined over the code points U+0000–U+2FFFF [31], so the backend bounds every synthesized character to $[U+0000, U+D7FF] \cup [U+E000, U+2FFFF]$, i.e., that alphabet intersected with the scalar values that Rust can hold. We re-measured both ends on Z3 4.15.4: `str.from_code` returns the empty string from 0x30000 upward. This costs completeness and not soundness; a witness we accept is still a witness, but a divergence that only appears above U+2FFFF is not found.

Threats to validity. Following the usual taxonomy [26, 37], we see four threats. The construct threat is that a marker measures an error condition that we wrote into the reference, not the error space of the standard, and that the correspondence between the standard library and Z3 (§4) is assumed rather than proven. The internal threat is that a test could be filed under the wrong marker; acceptance is on the named marker, whose id is computed by the same function on both sides, and replay must fire that same marker, which is what rules this out. The external threat is that we have two languages, one machine, one solver version, and one model, and we make no claim beyond them. The conclusion threat is that every coverage count is relative to a budget on one machine and to a single recorded run of a nondeterministic model; a timeout can only turn a covered marker into a miss, never produce a witness, and the run is content-addressed so that it replays, but a fresh run may cover a different set of markers.

9 Conclusion

RuSMT lets an author write the reference semantics of a language once, as an ordinary recursive Rust program that is both the conformance oracle and the source of one reachability query per marked branch. For TOML 1.1.0, the product is a generated suite covering 146 of 183 markers, with the 37 misses named. Running it against four production parsers found 12 inputs on which the parsers and the oracle did not all agree, and among them a defect in `smol-toml` that had not been reported before: an array-of-tables header closed by a single bracket is accepted. The other divergences are two defects that the maintainers already track and 9 inputs that test behaviour the standard leaves to the implementation.

We are also able to say what did not work. Lowering the whole semantics into one query and asking Z3 to invert it closes the loop for IMP in under a quarter of a second and produces nothing for TOML; this is the known cost of a whole-program encoding, and what this artifact adds is a measurement of how abruptly the cost arrives. Generation is therefore carried by a language model, and acceptance stays with the solver on the unmodified query. Whether a per-path encoding moves that frontier is the question we would take up next.

Data-Availability Statement

The framework, the two executable specifications, the generated suites, the differential-testing harness, and the recorded run behind every number in this paper (the per-marker ledger, the four-parser differential result, and all recorded proposer exchanges) are

archived at Zenodo [11]. The emitted SMT-LIB queries are not archived, since they regenerate deterministically from the specifications.

Acknowledgments

We thank the SpecOps 2026 reviewers, whose comments on the submitted version prompted the implementation comparison of §6, the removal of a fixed candidate table in favour of live proposer calls, and the reframing of §4.

References

- [1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emmina Torlak, and Abhishek Udupa. 2013. Syntax-Guided Synthesis. In *2013 Formal Methods in Computer-Aided Design (FMCAD 2013)*. IEEE, 1–8. doi:10.1109/FMCAD.2013.6679385
- [2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6*. Technical Report. Department of Computer Science, The University of Iowa. Available at <https://smt-lib.org/papers/smt-lib-reference-v2.6-r2021-05-12.pdf>.
- [3] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. Springer, 193–207. doi:10.1007/3-540-49059-0_14
- [4] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 114–129. doi:10.1109/SP.2014.15
- [5] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI 2008)*. USENIX Association, 209–224. <https://www.usenix.org/conference/osdi-08/klee-unassisted-and-automatic-generation-high-coverage-tests-complex-systems>
- [6] Edmund Clarke, Daniel Kroening, and Flavio Lerda. 2004. A Tool for Checking ANSI-C Programs. In *TACAS*. 168–176. doi:10.1007/978-3-540-24730-2_15
- [7] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008) (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24
- [8] Ecma International TC39. 2026. Test262: ECMAScript Conformance Test Suite. <https://github.com/tc39/test262>. Accessed 2026-08-25.
- [9] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex*. MIT Press. ISBN 9780262062756.
- [10] Haryadi S. Gunawi, Cindy Rubio-González, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Ben Liblit. 2008. EIO: Error Handling is Occasionally Correct. In *6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 207–222. <https://www.usenix.org/legacy/events/fast08/tech/gunawi.html>
- [11] Mehra Haghshenas and Meng Xu. 2026. Reproduction Package for Article 'RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer'. Zenodo. doi:10.5281/zenodo.22095547
- [12] Claude Jard and Thierry Jéron. 2005. TGV: Theory, Principles and Algorithms. *Int. J. Softw. Tools Technol. Transf.* 7, 4 (2005), 297–315. doi:10.1007/s10009-004-0153-x
- [13] Sumit Kumar Jha, Susmit Jha, Patrick Lincoln, Nathaniel D. Bastian, Alvaro Velasquez, Rickard Ewetz, and Sandeep Neema. 2023. Counterexample Guided Inductive Synthesis Using Large Language Models and Satisfiability Solving. In *IEEE Military Communications Conference (MILCOM 2023)*. IEEE. doi:10.1109/MILCOM58377.2023.10356332
- [14] Jinwoo Kim, Qinheping Hu, Loris D'Antoni, and Thomas W. Reps. 2021. Semantics-Guided Synthesis. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–32. doi:10.1145/3434311
- [15] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering (ICSE 2023)*. IEEE. doi:10.1109/ICSE48619.2023.00085
- [16] Yihe Li, Ruijie Meng, and Gregory J. Duck. 2025. Large Language Model Powered Symbolic Execution. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 385 (2025). doi:10.1145/3763163
- [17] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith J. C. Johnson, Thomas W. Reps, and Loris D'Antoni. 2024. Synthesizing Formal Semantics from Executable Interpreters. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024). doi:10.1145/3689724 Introduces the Syntactic tool.

- [18] Kunhang Lv, Yuhang Dong, Rui Han, Fuqi Jia, Feifei Ma, and Jian Zhang. 2026. LLM-Guided Quantified SMT Solving over Uninterpreted Functions. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 17 (2026), 14304–14312. doi:10.1609/aaai.v40i17.38445 Introduces the AquaForte framework.
- [19] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. 2024. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. arXiv:2401.08807. doi:10.48550/arXiv.2401.08807
- [20] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [21] Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. 2014. Lem: Reusable Engineering of Real-World Semantics. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*. ACM, 175–188. doi:10.1145/2628136.2628143
- [22] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE 2021)*. IEEE, 13–24. doi:10.1109/ICSE43902.2021.00015
- [23] Sebastian Poehlau and Aurélien Francillon. 2020. Symbolic Execution with SymCC: Don't Interpret, Compile!. In *USENIX Security*. 181–198. <https://www.usenix.org/conference/usenixsecurity20/presentation/poehlau>
- [24] Tom Preston-Werner et al. 2025. TOML: Tom's Obvious, Minimal Language, Version 1.1.0. <https://toml.io/en/v1.1.0>. Language specification.
- [25] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. doi:10.1016/j.jlap.2010.03.012
- [26] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. doi:10.1007/s10664-008-9102-8
- [27] Len Sassaman, Meredith L. Patterson, Sergey Bratus, and Michael E. Locasto. 2013. Security Applications of Formal Language Theory. *IEEE Systems Journal* 7, 3 (2013), 489–500. doi:10.1109/JSYST.2012.2222000
- [28] Nicolas Seriot. 2016. Parsing JSON is a Minefield. http://seriot.ch/parsing_json.php. Accessed 2026-06-20.
- [29] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Tom Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: Effective Tool Support for the Working Semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*. ACM, 1–12. doi:10.1145/1291151.1291155
- [30] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit A. Seshia, and Vijay A. Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2006)*. ACM, 404–415. doi:10.1145/1168857.1168907
- [31] Cesare Tinelli, Clark Barrett, and Pascal Fontaine. 2020. SMT-LIB Theory of Unicode Strings. <https://smt-lib.org/theories-UnicodeStrings.shtml>. Accessed 2026-08-24.
- [32] Emina Torlak and Rastislav Bodík. 2013. Growing Solver-Aided Languages with Rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward! 2013)*. ACM, 135–152. doi:10.1145/2509578.2509586
- [33] Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*. ACM, 530–541. doi:10.1145/2594291.2594340
- [34] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhm. 2025. Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation. arXiv preprint arXiv:2504.17542 (2025). arXiv:2504.17542 doi:10.48550/arXiv.2504.17542 To appear, IEEE Symposium on Security and Privacy (S&P) 2026.
- [35] Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. 2024. Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification. In *Computer Aided Verification (CAV 2024) (Lecture Notes in Computer Science)*. Springer. doi:10.1007/978-3-031-65630-9_16 arXiv:2404.00762.
- [36] Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press. ISBN 9780262231695. doi:10.7551/mitpress/3054.001.0001
- [37] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer. ISBN 9783642290435. doi:10.1007/978-3-642-29044-2
- [38] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *International Conference on Learning Representations (ICLR 2024)*. arXiv:2310.04870 https://proceedings.iclr.cc/paper_files/paper/2024/hash/0c86142265c5e2c900613dd1d031cb90-Abstract-Conference.html
- [39] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*. ACM, 283–294. doi:10.1145/1993498.1993532
- [40] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. 2014. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 249–265. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/yuan>

Received 2026-07-01; accepted 2026-07-16